

Data Engineer Interview Prep — Doc

3

Oracle warehouse + hybrid lakehouse (Iceberg, OCI, Parquet, external tables)

Q&A + prep focus per area

Tailored to a JD focused on: **enterprise Oracle data warehouse** (data flow/transformation patterns, external tables, flat files, high-volume batch ingestion), **performance optimization** (storage/query/orchestration), **cloud-to-Oracle integration**, **data catalog**, **shell-script orchestration**, **Oracle-native transform tools**, a **hybrid lakehouse with Apache Iceberg**, **OCI Object Storage**, **Parquet**, **external tables**, **Linux admin/tuning**, and acting as a **thought partner** (plus future **MDM / golden record**). Each section: **prep focus** + **Q&A**.

SECTION 1 — Enterprise Oracle Data Warehouse

Prep focus: Oracle warehouse patterns — external tables, flat-file loads, high-volume batch ingestion, partitioning, and transformation patterns (ETL/ELT with PL/SQL).

Q: How do you ingest high-volume flat files into Oracle?

- **External tables** — map a flat file (in a directory/object store) as a read-only table and `INSERT ... SELECT` from it (great for ELT; no separate load step).
- ***SQLLoader*** — high-speed bulk loader for flat files (direct-path loads).
- **Data Pump** for Oracle-to-Oracle. For very large loads, use **direct-path** + **partition exchange** for near-instant publishing.

Q: What is an external table and when do you use it? A table whose data lives **outside** the database (a flat file or object store) that Oracle reads on demand. Use it to query/ingest files without loading them first — the foundation of an ELT/lakehouse pattern (and now Iceberg/Parquet in object storage).

Q: Explain data flow / transformation patterns in an Oracle DW. Typical **ELT**: land raw (external tables/staging) → transform with **PL/SQL / SQL / MERGE / materialized views** into staging → load **dimensional or normalized** target tables → publish via **partition exchange**. Orchestrated by shell scripts/scheduler, with logging and restartability.

Q: How do you do incremental loads and upserts in Oracle? `MERGE` (upsert) on a key, **CDC** or watermark columns for deltas, and **partition exchange** to swap in new data. Track SCD Type 2 with effective/expiry dates.

SECTION 2 — Performance Optimization (storage, query, orchestration)

Prep focus: the JD stresses tuning a long-standing production warehouse — know partitioning, indexing, stats, parallelism, materialized views, and reading execution plans.

Q: How do you optimize query performance in Oracle?

- **Partitioning** (range by date, list, hash) → **partition pruning** so queries scan less.
- **Indexes** — B-tree for selective lookups, **bitmap** for low-cardinality DW columns.
- **Gather statistics** (`DBMS_STATS`) so the optimizer picks good plans.
- Read the **execution plan** (`EXPLAIN PLAN` / `AUTOTRACE`), look for full scans, bad joins.

- **Parallel query/DML** for big scans; **materialized views** to pre-aggregate.
- Avoid functions on indexed columns; use bind variables; partition-wise joins.

Q: Storage-layer optimizations? Right partitioning + compression (Advanced/HCC), correct tablespaces, avoid row chaining/ migration, manage indexes, and archive/purge old partitions. Size for I/O.

Q: How do you tune orchestration for high-volume batch? Parallelize independent jobs, load in **partitions** and swap via **partition exchange**, disable/rebuild indexes around bulk loads, use **direct-path** loads, checkpoint/restartable steps, and schedule to spread I/O.

Q: How would you approach a slow, long-standing warehouse with lots of legacy logic? Measure first (AWR/plans/wait events), find the worst offenders, tune the biggest wins (partitioning, stats, indexes, rewriting the hot queries/MVs), and change carefully with regression testing — respecting the accumulated business logic. Document as you go.

Q: Materialized views — why and how? Pre-computed query results (often aggregates) that refresh (full/fast/incremental) — speed up repeated heavy queries; **query rewrite** lets the optimizer use them automatically.

SECTION 3 — Oracle-Native Transform Tools & PL/SQL

Q: What Oracle-native transformation tools do you know? **PL/SQL** (procedures/packages for ETL logic), **SQL MERGE**, **materialized views**, **external tables**, *SQLLoader, and Oracle Data Integrator (ODI)** for ELT. Modern: evaluating Oracle-native transform tooling as the JD mentions.

Q: When PL/SQL vs a set-based SQL approach? Prefer **set-based SQL** (one big **INSERT/MERGE**) for performance; use PL/SQL for procedural control, error handling, batching, and orchestration. Avoid row-by-row loops on big data.

SECTION 4 — Cloud-to-Oracle Integration & Data Catalog

Q: How do you integrate cloud object storage with Oracle? Use **external tables over object storage** (OCI Object Storage / S3) so Oracle reads Parquet/ flat files directly; **DBMS_CLOUD** (Autonomous DB) to load/query object-store data; or land files in object storage and ingest via external tables. This is the bridge to the lakehouse.

Q: What is a data catalog and why maintain one? A central inventory of datasets — tables, schemas, locations, owners, lineage — so teams can find and trust data. (OCI Data Catalog / Glue Data Catalog.) It's the metadata backbone for both the warehouse and the lake.

SECTION 5 — Shell-Script Orchestration & Linux Admin

Prep focus: the JD includes shell-based jobs and light Linux admin/tuning — be comfortable with bash and basic performance tools.

Q: How do shell scripts orchestrate ETL jobs? Bash scripts scheduled via **cron** run SQL*Loader/SQL scripts in sequence, check return codes, log output, send alerts on failure, and handle restarts. Common pattern: driver script → dependent steps → status/error handling.

Q: Key bash/Linux skills for ETL ops? File handling (`awk` / `sed` / `grep`), scheduling (`cron`), process control, exit-code checks, logging, and moving files to/from object storage (`oci` / `aws` CLI).

Q: How do you do light Linux performance tuning on ETL servers? Monitor with `top` / `htop`, `vmstat`, `iostat`, `free`, `df`; find CPU/memory/IO bottlenecks; tune file descriptors/limits, disk/mount options, and job concurrency; check logs; manage disk space and log rotation.

SECTION 6 — Hybrid Lakehouse (Apache Iceberg, OCI Object Storage, Parquet, External Tables)

Prep focus: this is the modern direction of the role — know Iceberg and the external-table lakehouse pattern.

Q: What is a hybrid lakehouse and why move toward it? Combine the **warehouse** (Oracle, curated, fast SQL) with a **lake** (object storage + open formats) so you get cheap, scalable storage and open access **plus** warehouse-grade querying. It reduces cost, avoids lock-in, and prepares for cloud-object-storage ingestion.

Q: What is Apache Iceberg and why use it? An open **table format** for huge analytic datasets on object storage. It adds **ACID transactions, schema/partition evolution, time travel, and hidden partitioning** on top of Parquet files — so a data lake behaves like a reliable warehouse table, readable by many engines (Spark, Trino, Oracle external tables, etc.).

Q: Iceberg vs plain Parquet external tables? Plain Parquet = files only (no transactions, manual partitions). **Iceberg** adds a metadata layer for ACID, safe concurrent writes, schema/partition evolution, and time travel — much better for evolving, high-volume tables.

Q: How would you design the ingestion shift to cloud object storage + data lake? Land source data as **Parquet in OCI Object Storage** (raw), define **Iceberg/external tables** over it, transform into curated tables, and expose to both Oracle (external tables) and lake engines. Keep raw immutable, partition by date, and catalog everything — a medallion-style lakehouse.

Q: Iceberg vs Delta vs Hudi? All are open table formats giving ACID/upserts/time-travel on object storage. **Iceberg** is engine-neutral and strong on schema/partition evolution (and what this role uses); **Delta** is Databricks-centric; **Hudi** is strong for streaming upserts.

SECTION 7 — Thought Partnership & MDM / Golden Record

Prep focus: the JD wants a proactive partner, and future master-data work.

Q: What is Master Data Management (MDM) and a "golden record"? MDM creates a single, trusted version of core business entities (customer, product, vendor) by **matching, merging, and**

de-duplicating records from many sources. The **golden record** is that consolidated, authoritative record. It needs solid data foundations (clean, cataloged, governed data) first — which is why it's a later workstream.

Q: How do you act as a "thought partner" rather than wait for direction? Understand the business goal, propose options with trade-offs, surface patterns and risks early, prototype, and communicate clearly with senior stakeholders — bring recommendations, not just questions.

Q: How do you approach modernizing a legacy environment? Respect existing business logic, measure before changing, deliver incremental wins, keep raw/ source-of-truth intact, document, and align changes to business value and risk.

QUICK CHECKLIST (this JD)

- [] Oracle ingestion: **external tables**, *SQLLoader, partition exchange**, **MERGE**.
 - [] Oracle tuning: **partitioning**, indexes (bitmap for DW), **DBMS_STATS**, plans, parallel, **materialized views**.
 - [] Cloud-to-Oracle: external tables over object storage, DBMS_CLOUD, data catalog.
 - [] **Shell/cron orchestration** + basic **Linux tuning** (top/iostat/vmstat).
 - [] **Hybrid lakehouse: Apache Iceberg + OCI Object Storage + Parquet** + external tables; Iceberg vs Delta vs Hudi.
 - [] PL/SQL vs set-based SQL; ELT patterns.
 - [] **MDM / golden record** basics; being a proactive thought partner.
-